Entrega parcial

- Aluno(a): Vitor Mateus Weirich (weirichvitor@gmail.com)
- Código: 369881
- Turma: EAD54-12
- Professor(a): Alysson Borges
- Disciplina: 30062 - PROJETO INTEGRADOR - EAD54-12

Essa pasta contém todos os arquivos referentes a entrega parcial, incluindo:

- Canvas do modelo de negocio
- Diagramas
- Requisitos e escolha de plataformas/tecnologias
- Protótio do app mobile

Segue link para o github do projeto, contendo esse documento, além de todos os arquivos separados: https://github.com/vitorweirich/ads-projeto-integrador

Canvas do Modelo de Negócio

Criado para: Plataforma de Compartilhamento Temporário de Arquivos

Criado por: Vitor Mateus Weirich

Em: Dia Mês Ano 08/03/2026

Iteração: No. 2

Parcerias-Chave

Quem é fundamental para o modelo de negócio prosperar?

- Provedores de nuvem (AWS, Google Cloud, Azure).

- Gateways de pagamento (Stripe, PayPal, etc.).

- Agências de marketing digital.

- Parceiros de mídia e influenciadores para divulgação.

Atividades-Chave

Quais Atividades-Chave são necessárias para entregar nossa(s) Proposta(s) de Valor?

- Desenvolver e manter a plataforma.

- Garantir segurança e desempenho no envio de arquivos.

- Gerenciar servidores e processos de exclusão automática.

- Promover o serviço e captar novos usuários.

Recursos-Chave

Que recursos necessitamos para entregar uma proposta de valor adequada ?

- Infraestrutura de armazenamento em nuvem escalável.

- Sistema de exclusão automática e controle de tempo.

- Equipe técnica de desenvolvimento e suporte.

- Plataformas web, mobile e desktop integradas.

Propostas de Valor

Qual valor entregamos aos clientes?
Quais problemas dos clientes estamos ajudando a resolver?

- Compartilhamento simples e rápido de arquivos grandes.

.
- Armazenamento temporário com exclusão automática.

- Custos menores por uso otimizado do espaço.

- Alternativa prática a ferramentas limitadas (ex: WhatsApp, e-mail, discord etc).

Relacionamentos com Clientes

Como é o nosso relacionamento com os diferentes segmentos de clientes?

- Interface intuitiva e Suporte automatizado.

- Atendimento rápido via chat ou e-mail.

- Comunicação transparente sobre prazos e limites.

- Incentivo à fidelização por meio de planos premium.

Canais

Como entregamos a nossa proposta de valor para os nossos clientes?
Que meios físicos os alcançamos?

- Site responsivo acessível via navegador.

- Aplicativo mobile (Android e iOS). Aplicativo desktop para upload de grandes volumes.

- Divulgação via redes sociais e campanhas digitais.

Segmentos de Clientes

Para quem estamos criando valor?
Quem são nossos clientes mais importantes?
Que persona representa o nosso cliente?

- Usuários que precisam enviar arquivos grandes com facilidade.

- Profissionais de mídia e audiovisual.

- Equipes remotas que trocam arquivos de forma rápida e segura.

- Pessoas que desejam compartilhar arquivos
sem ocupar armazenamento permanente.

- Empresas cujo workflow depende do envio de arquivos regularmente. Ex: Compartilhamento de arquivo, artefatos em geral.

Estrutura de Custo

Quais são os custos mais importantes no modelo de negócio?

- Custos de servidores e armazenamento temporário.

- Desenvolvimento e manutenção da plataforma.

- Suporte ao cliente e marketing digital.

- Licenças, APIs e integrações externas.

Fontes de Renda

Liste suas fontes de renda.
Como o(s) segmento(s) de cliente(s) paga(m) pelos serviços e produtos oferecidos?

- Planos pagos com maior limite de tamanho e tempo de armazenamento.

- Versão gratuita com limite reduzido e publicidade.

- Parcerias com empresas para envio corporativo de arquivos.

- Assinaturas mensais ou anuais.

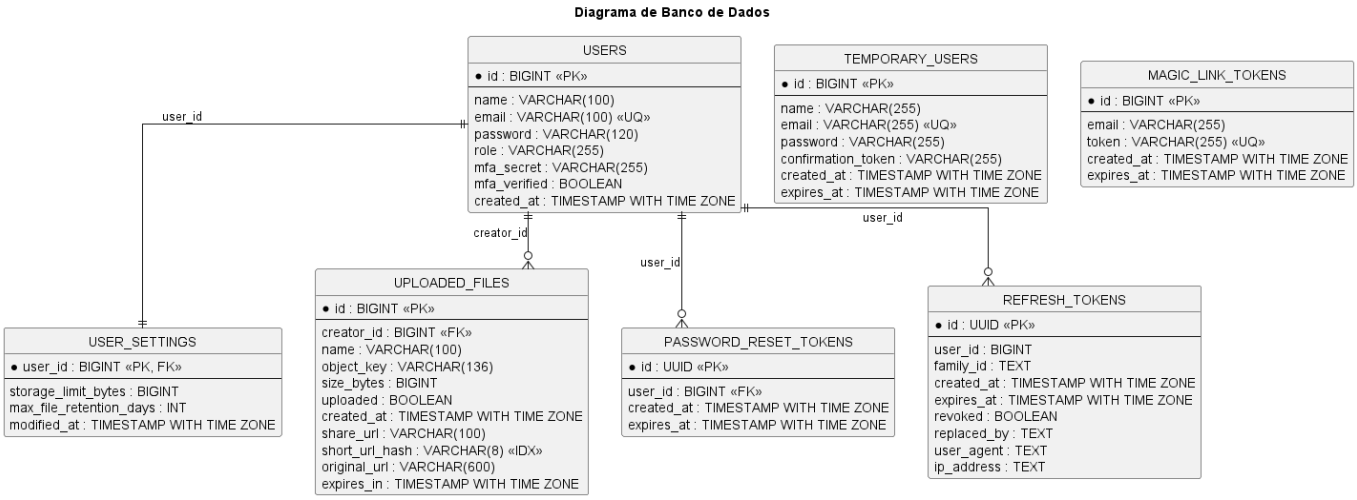
- Combrança por uso de apis (integração em outros sistemas existens).

This work is licensed under the Creative Commons Attribution-Share Alike 3.0 Unported License.
To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/3.0/
or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.

Diagramas do Projeto

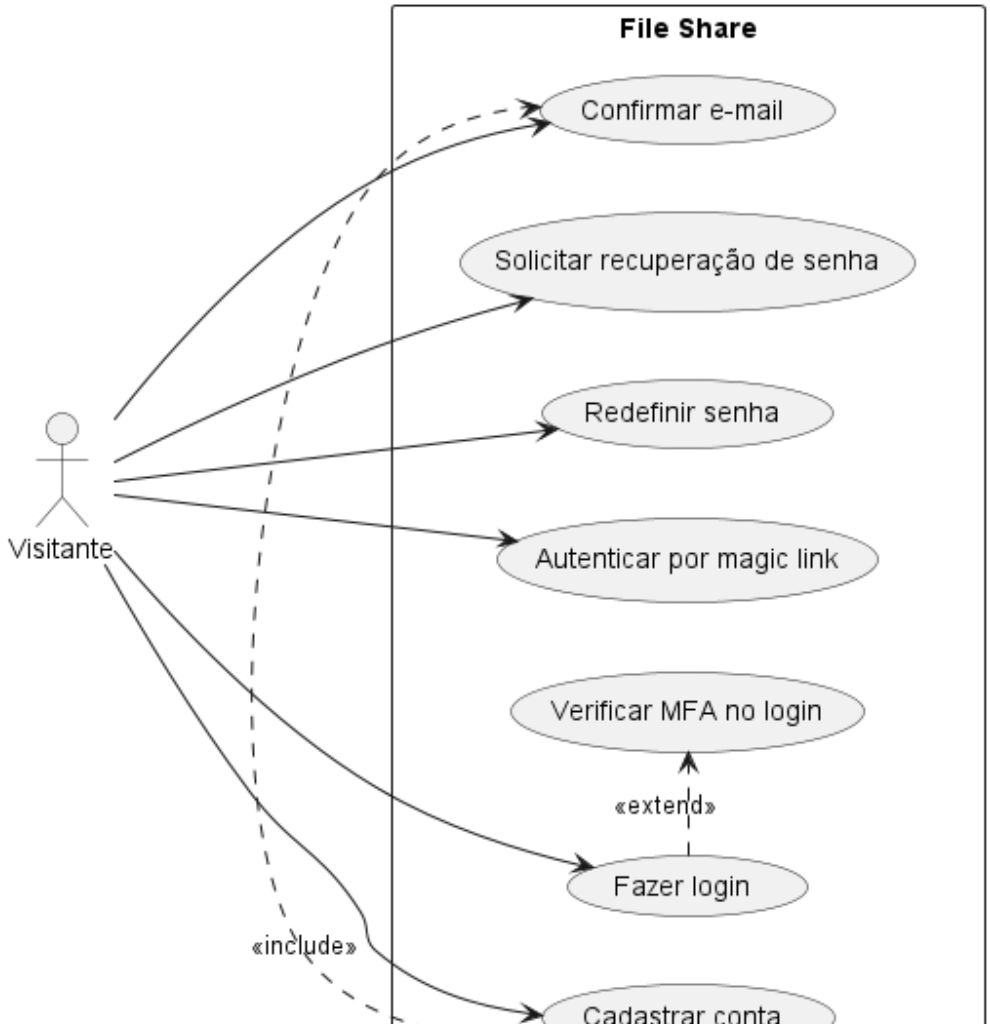
Este documento centraliza os diagramas em formato PNG.

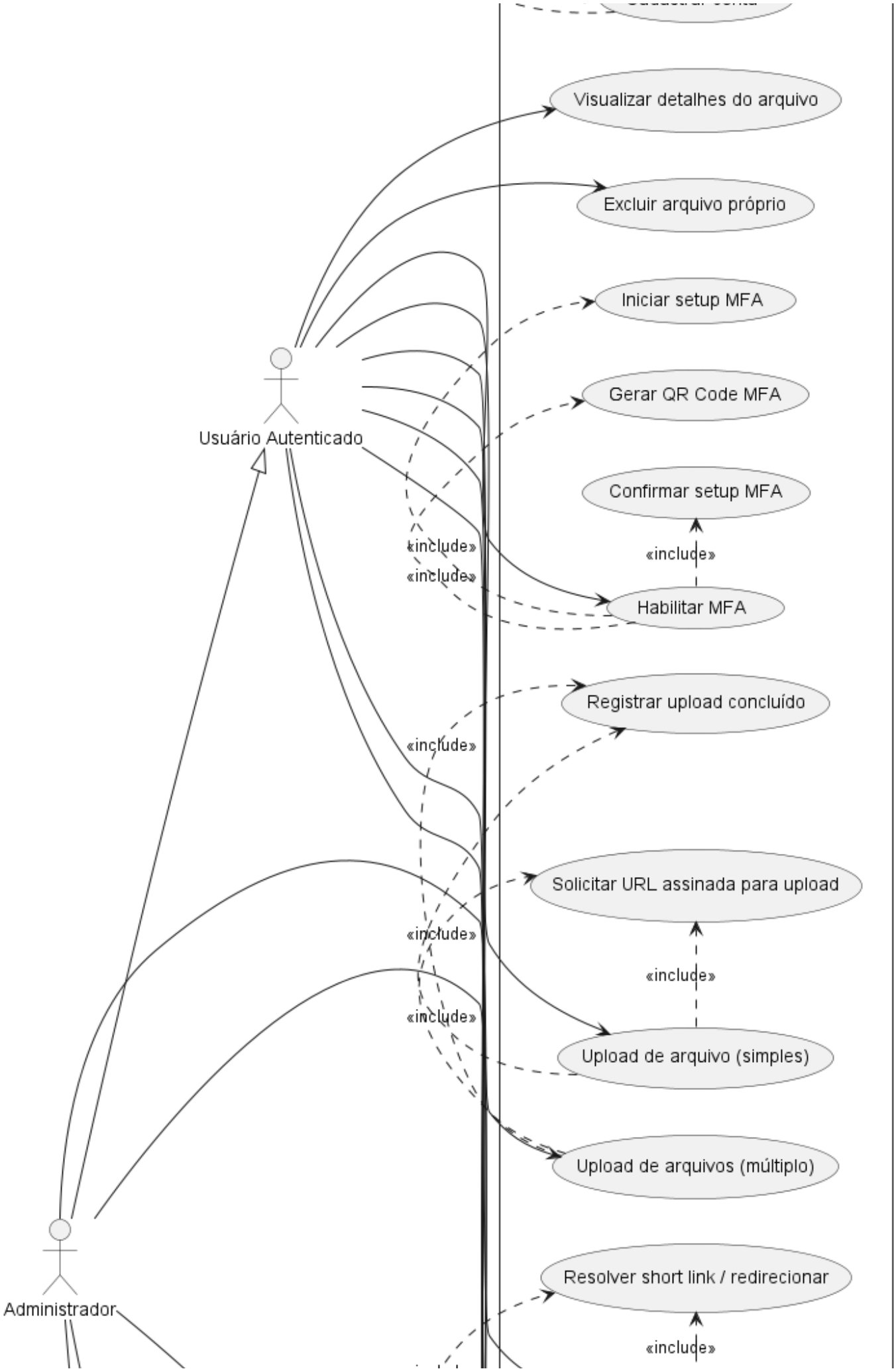
Diagrama do Banco de Dados

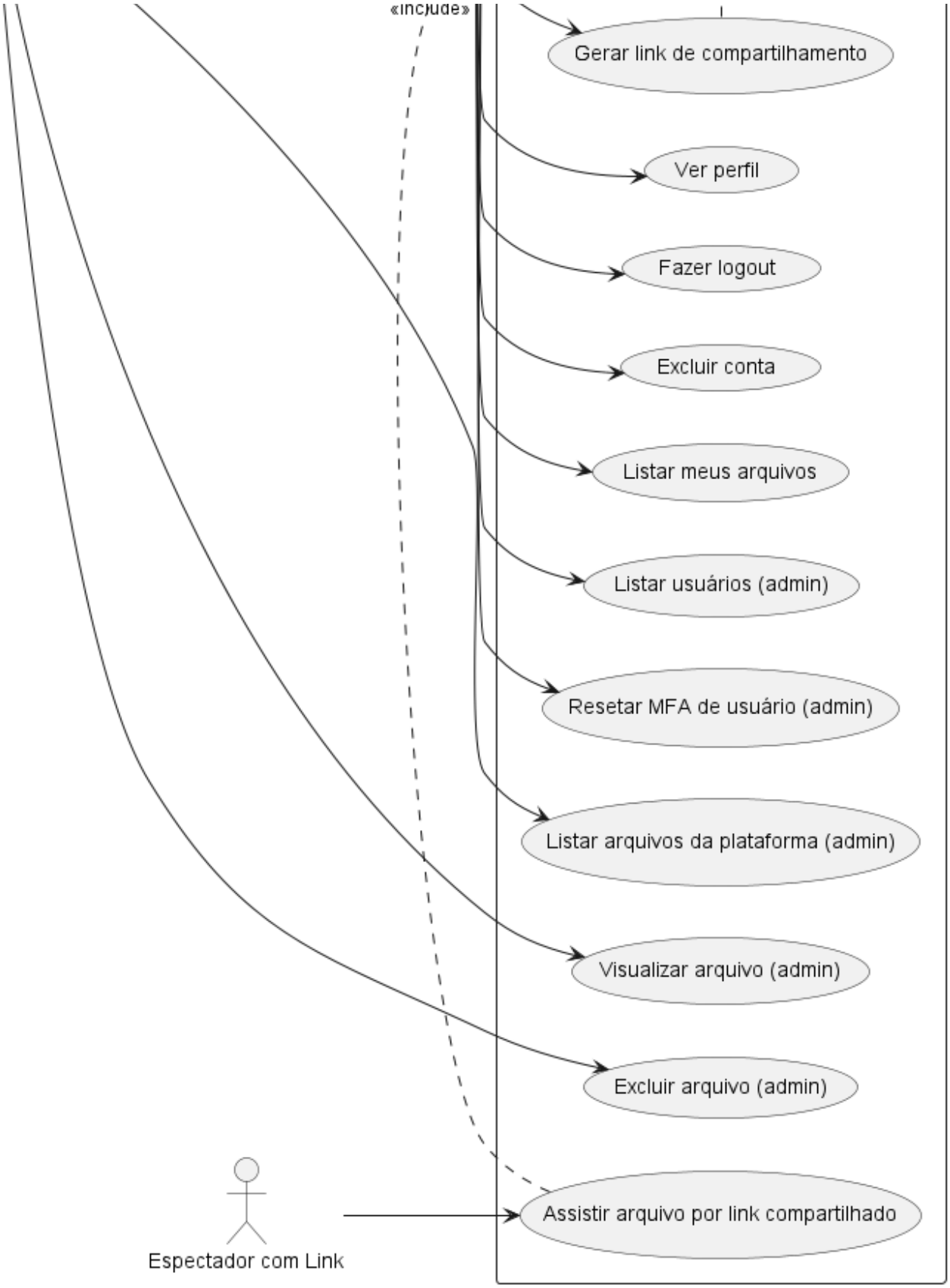


Caso de Uso

Diagrama de Caso de Uso

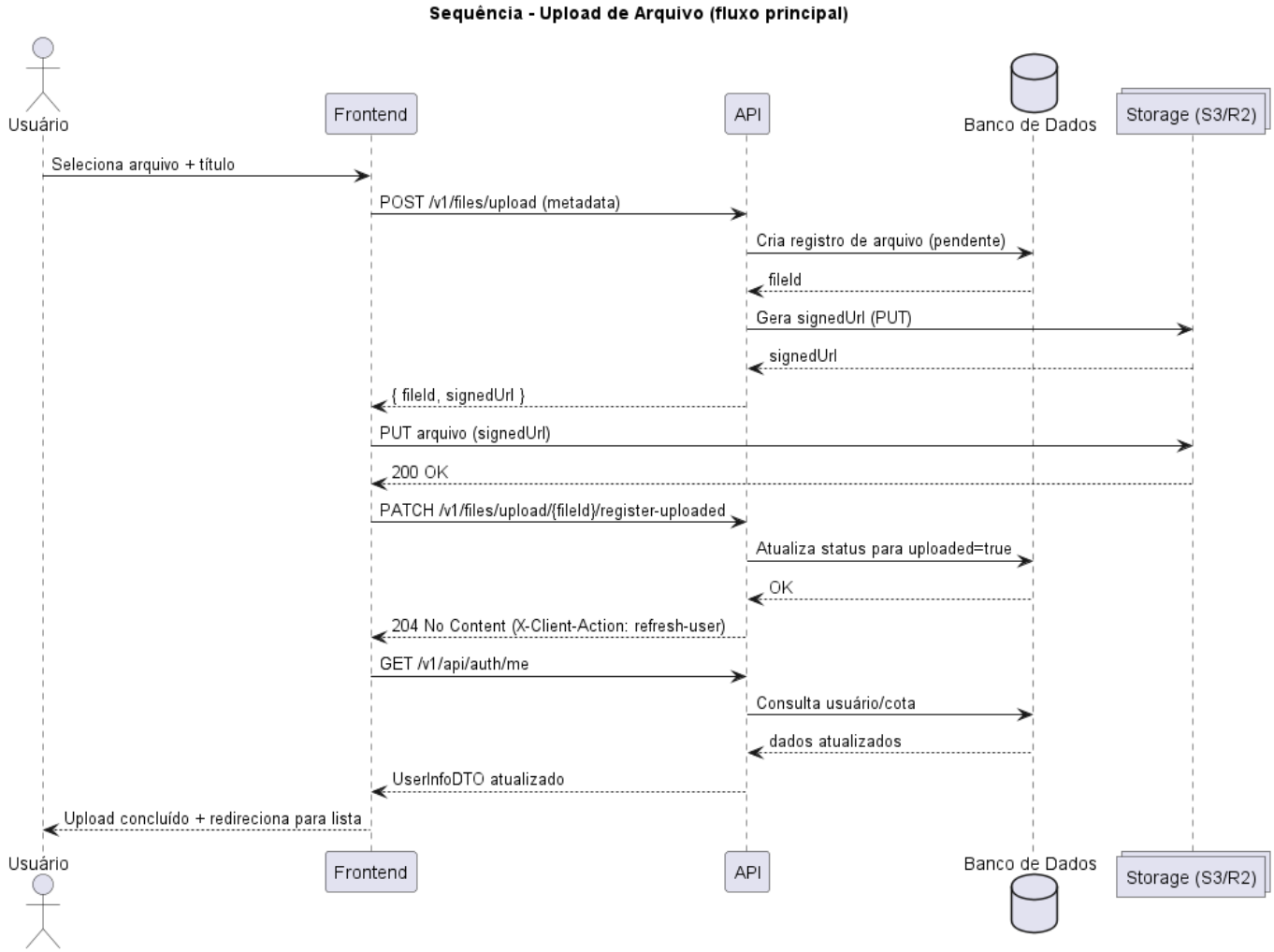



Visualizar detalhes do arquivo
Excluir arquivo próprio
Iniciar setup MFA
Gerar QR Code MFA
Usuário Autenticado
Confirmar setup MFA
«include»
«include»
«include»
Habilitar MFA
Registrar upload concluído
«include»
Solicitar URL assinada para upload
«include»
«include»
«include»
Upload de arquivo (simples)
Upload de arquivos (múltiplo)
Resolver short link / redirecionar
Administrador
«include»

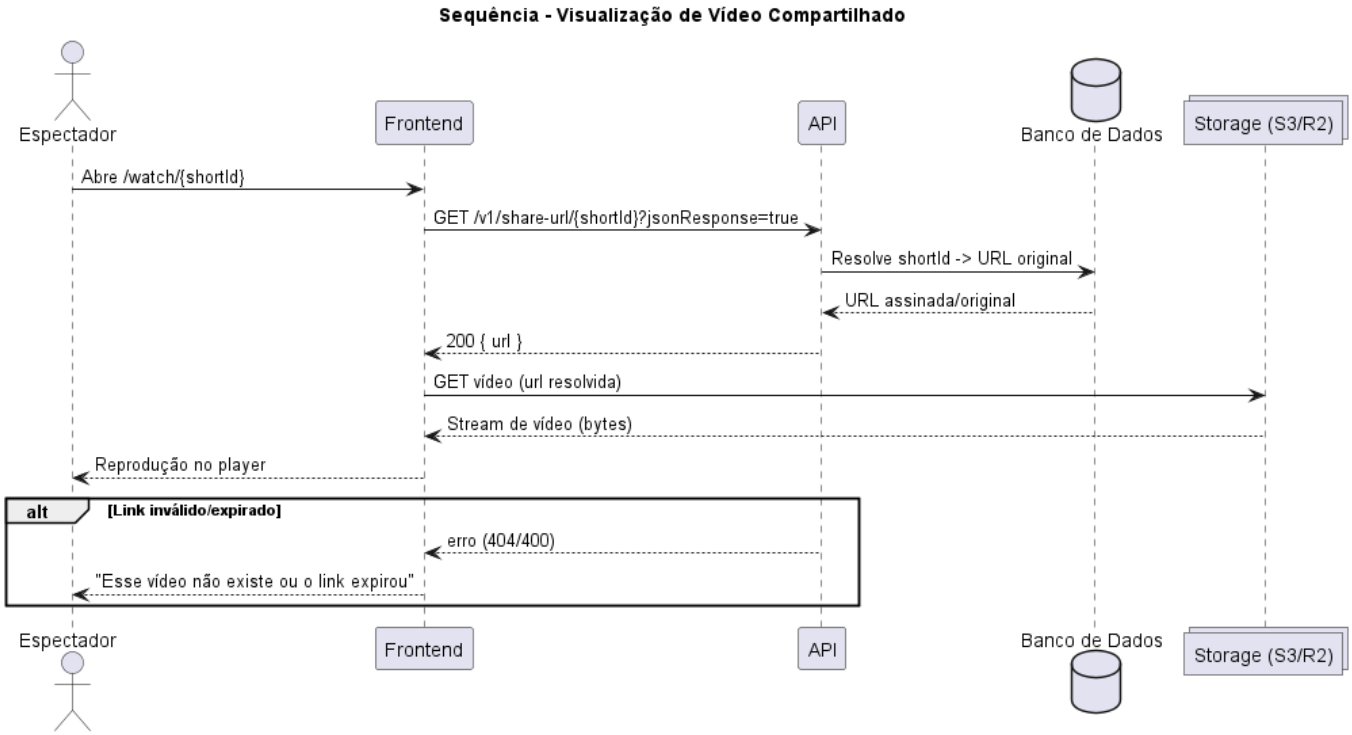


Diagramas de Sequencia

Sequencia - Upload de Arquivo



Sequencia - Visualizacao de Arquivo



Diagramas de Arquitetura

Diagrama de Pacotes (Arquitetura da Aplicacao)

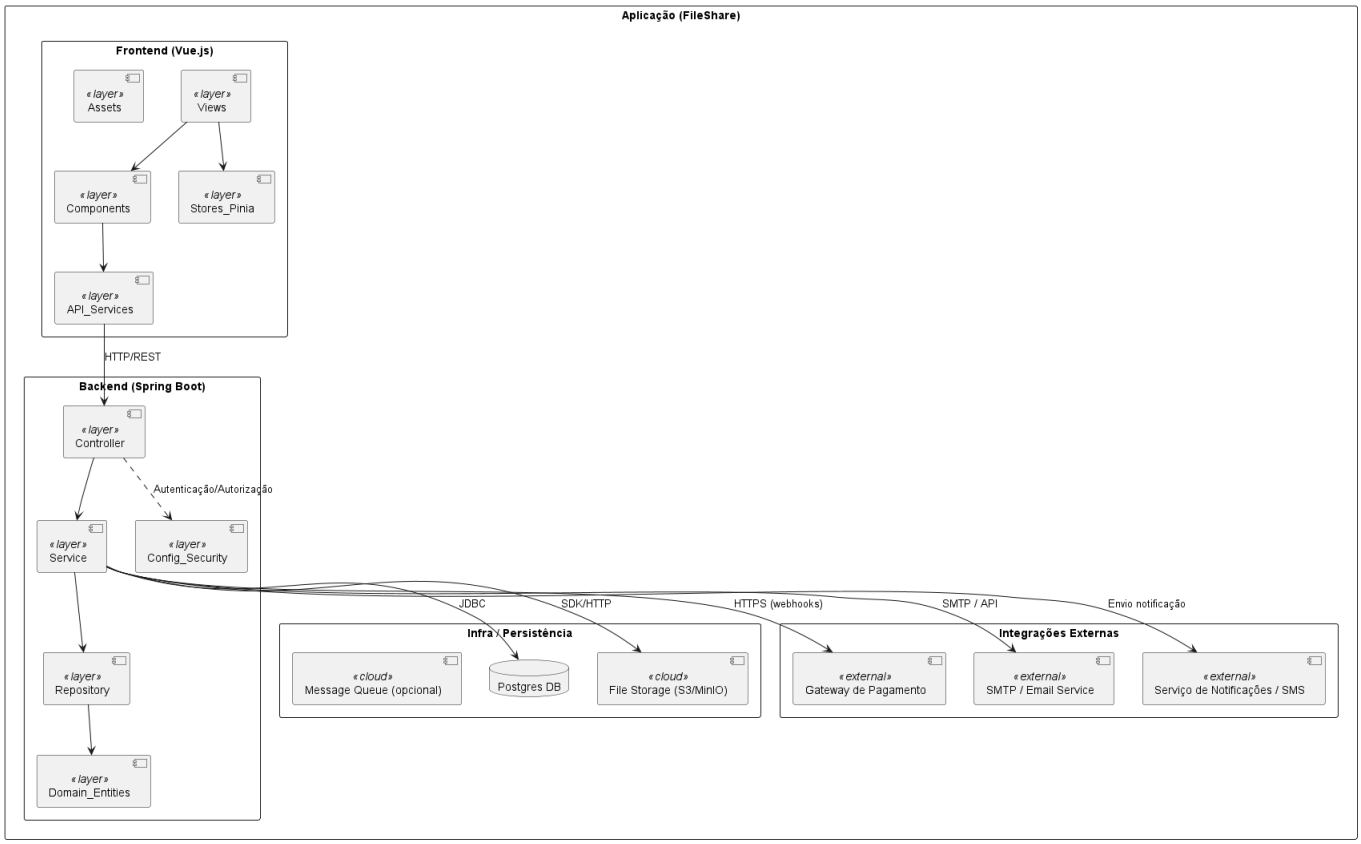
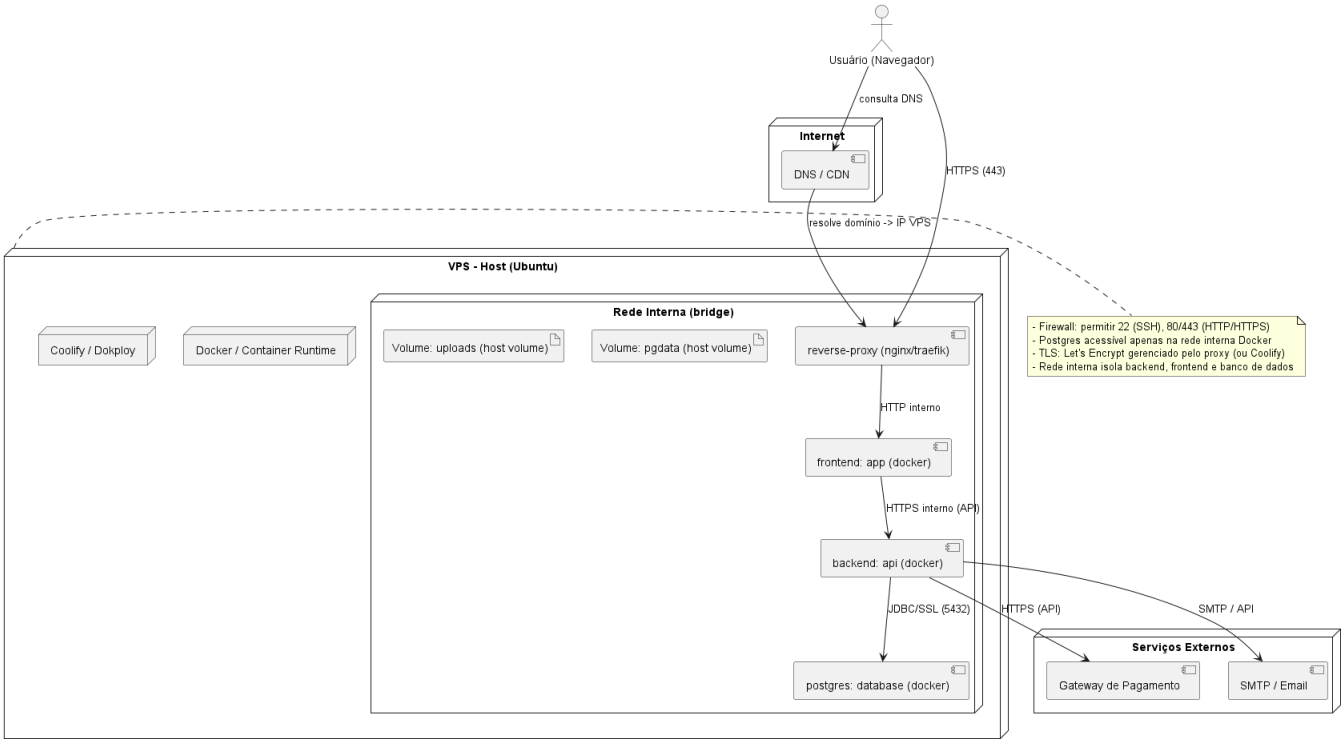
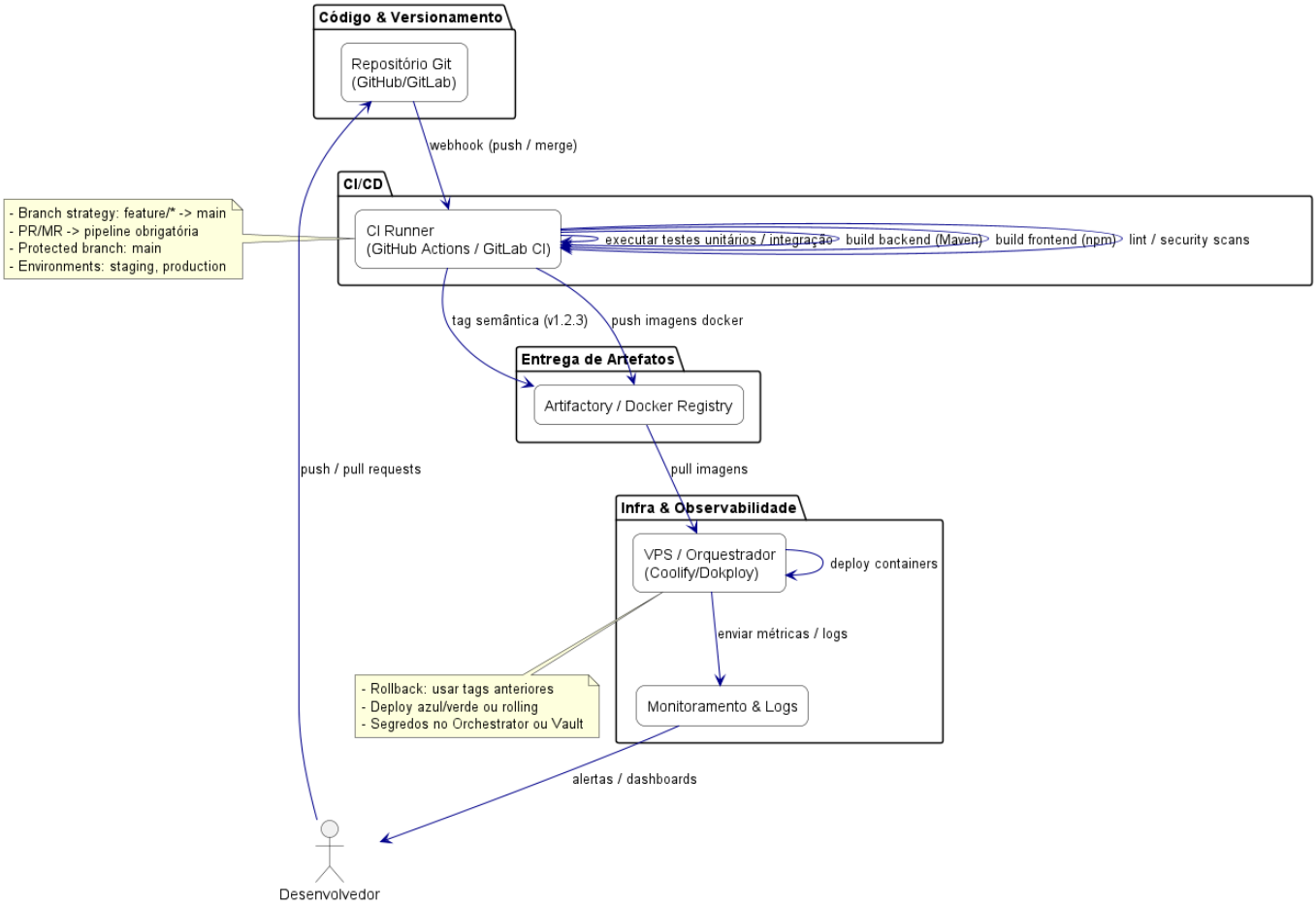


Diagrama de Implantacao (Deployment)



Arquitetura DevOps (CI/CD)


Código & Versionamento
Repositório Git
(GitHub/GitLab)
webhook (push / merge)
CI/CD
- Branch strategy: feature/* -> main
- PR/MR -> pipeline obrigatória
- Protected branch: main
- Environments: staging, production
CI Runner
(GitHub Actions / GitLab CI)
executar testes unitários / integração
build backend (Maven)
build frontend (npm)
lint / security scans
tag semântica (v1.2.3)
push imagens docker
Entrega de Artefatos
Artifactory / Docker Registry
push / pull requests
pull imagens
Infra & Observabilidade
VPS / Orquestrador
(Coolify/Dokploy)
deploy containers
enviar métricas / logs
- Rollback: usar tags anteriores
- Deploy azul/verde ou rolling
- Segredos no Orchestrator ou Vault
Monitoramento & Logs
alertas / dashboards
Desenvolvedor

Projeto Aplicativo Mobile: FileShare

1. Requisitos do Aplicativo

1.1. Requisitos Funcionais

- **Autenticação de Usuário:**
  - **Login:** Permitir que usuários existentes façam login com e-mail e senha.
  - **Cadastro:** Permitir que novos usuários se cadastrem fornecendo e-mail e senha (com confirmação de senha).
- **Gerenciamento de Arquivos:**
  - **Listagem de Arquivos:** Exibir uma lista dos Arquivos do usuário (Meus Arquivos).
  - **Visualização de Arquivos:** Permitir a visualização de um arquivo selecionado, exibindo título e visualização se existir (nem todos os tipos de arquivos terão como serem visualizados).
  - **Envio de Arquivo:** Permitir que o usuário selecione um arquivo do dispositivo, forneça um título e o envie para a plataforma.

1.2. Requisitos Não Funcionais

- **Usabilidade:** A interface deve ser intuitiva e fácil de usar.
- **Desempenho:** O aplicativo deve ser responsivo e carregar informações rapidamente.
- **Segurança:** Credenciais e dados dos usuários devem ser protegidos.
- **Compatibilidade:** O aplicativo deve ser compatível com iOS e Android.

2.Escolha da Plataforma de Desenvolvimento e Tecnologias

2.1. Plataforma do Aplicativo Mobile: React Native

- **Desenvolvimento Multiplataforma:** Permite criar aplicativos para iOS e Android a partir de uma única base de código, reduzindo tempo e custos de desenvolvimento. Como o objetivo é atingir o maior número de usuários, essa é a melhor escolha.
- **Familiaridade com JavaScript/TypeScript:** A equipe (no caso, eu) já possui experiência com essas linguagens, diminuindo a curva de aprendizado.
- **Desempenho Próximo ao Nativo:** Embora seja uma estrutura híbrida, o React Native utiliza componentes nativos. Apesar de não alcançar a performance de um desenvolvimento totalmente nativo, considerando a falta de experiência em iOS e Android nativos, essa abordagem tende a resultar em um desempenho melhor no projeto.
- **Ecossistema Rico e Comunidade Ativa:** A vasta quantidade de bibliotecas e a comunidade ativa facilitam o desenvolvimento e a resolução de problemas.
- **Componentização:** A arquitetura baseada em componentes favorece modularidade, reuso de código e manutenção mais simples, além de facilitar a evolução do aplicativo.

2.2. Plataforma do Site (Frontend Web): Vue.js

Para o desenvolvimento do site, foi escolhida a plataforma **Vue.js**:

- **Desenvolvimento Rápido e Produtivo:** Vue.js é conhecido por sua simplicidade e curva de aprendizado baixa, permitindo prototipação e desenvolvimento ágil.
- **Componentização:** Assim como React Native, Vue.js favorece modularidade e reuso de código.
- **Ecossistema e Comunidade:** Possui uma comunidade ativa e diversas bibliotecas para facilitar integrações e funcionalidades.
- **Performance:** Vue.js oferece boa performance para aplicações SPA (Single Page Application).

> E claro, além de tudo isso, o Vue.js ajuda a evitar aquela dor de cabeça de lidar com o React (quem nunca se perdeu nos hooks que atire a primeira pedra!).

2.3. Plataforma do Backend (API): Spring Boot

Para a API backend, foi escolhido o **Spring Boot**:

- **Robustez e Escalabilidade:** Spring Boot é uma plataforma madura para desenvolvimento de APIs REST, com suporte a escalabilidade e segurança.
- **Integração com Banco de Dados:** Facilita integração com bancos relacionais como PostgreSQL.
- **Documentação e Comunidade:** Ampla documentação e comunidade ativa.
- **Produtividade:** Permite desenvolvimento rápido de APIs com configuração simplificada.
    - **Gosto pessoal e conhecimento:** A escolha do Spring Boot também foi motivada pelo gosto pessoal e familiaridade com o ecossistema Spring, o que facilita o desenvolvimento e manutenção da API.

2.4. Plataforma de Armazenamento de Arquivos: Object Storage S3

Para o armazenamento de arquivos, será utilizado um serviço compatível com a API S3 (Amazon S3, Cloudflare R2 ou MinIO):

- **Escalabilidade:** Permite armazenar grandes volumes de arquivos de forma eficiente.
- **Compatibilidade:** A API S3 é padrão de mercado, facilitando integrações.
- **Segurança:** Oferece recursos de controle de acesso e proteção de dados.
- **Custo:** Opções como MinIO permitem implantação local, reduzindo custos.

2.5. Banco de Dados: PostgreSQL

Para o armazenamento dos dados estruturados, será utilizado o **PostgreSQL**:

- **Confiabilidade e Robustez:** PostgreSQL é um dos bancos de dados relacionais mais confiáveis e utilizados no mercado.
- **Recursos Avançados:** Suporte a transações, integridade referencial, extensões e tipos de dados avançados.
- **Open Source:** Sem custos de licença, com grande comunidade e documentação.
- **Integração:** Fácil integração com Spring Boot e compatível com serviços de armazenamento em nuvem.

Projeto Aplicativo Mobile: FileShare

Disclaimer:

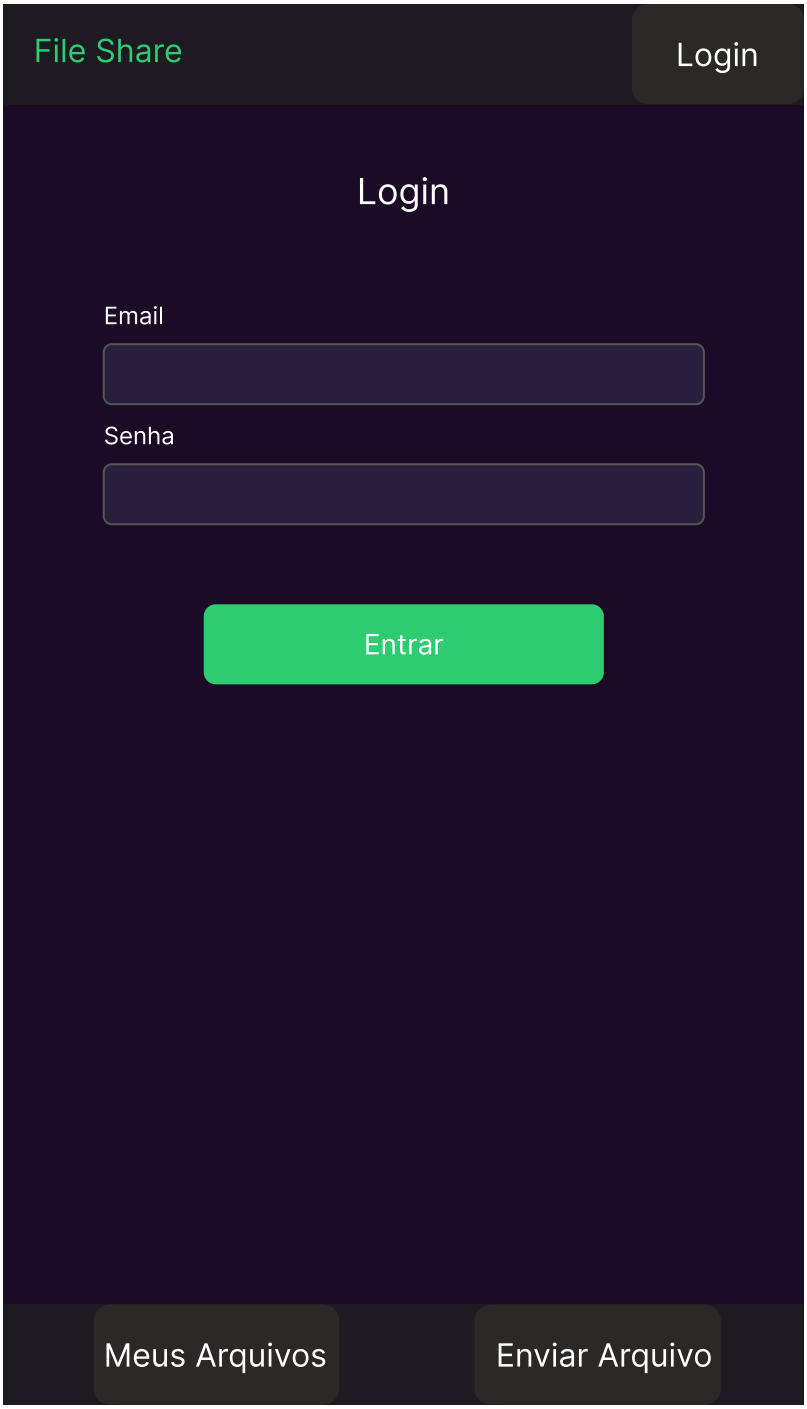
> **Os protótipos estão simplificados, servindo apenas para ter uma ideia da concepção do APP, mas não englobam todas as fucionalidades (a título de exemplos, só existem telas para quando o compartilhamento for um vídeo).**

1. Protótipo: Wireframes/Mockups

A seguir, os mockups que ilustram a interface e as principais funcionalidades do **FileShare**.

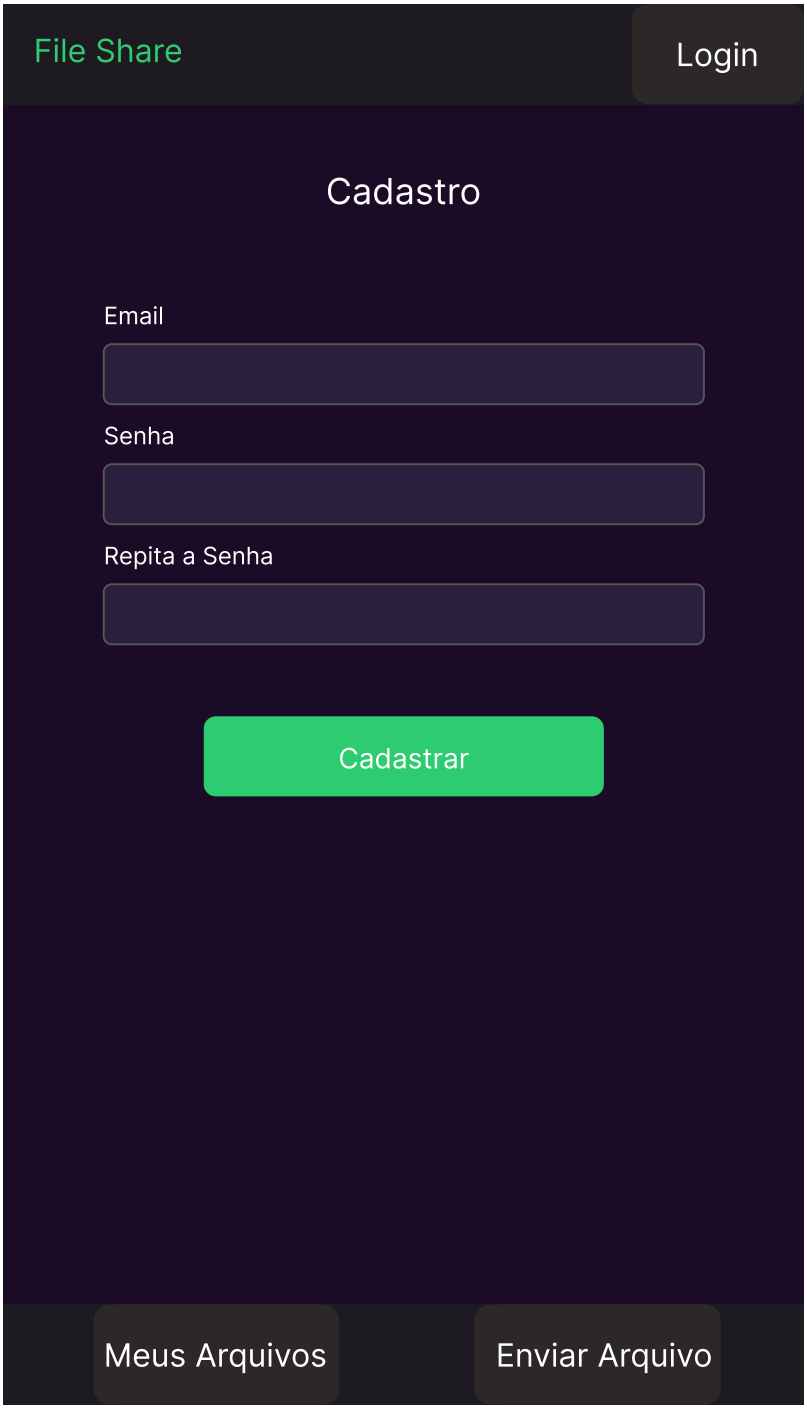
1.1. Tela de Login

Permite que os usuários existentes acessem suas contas.



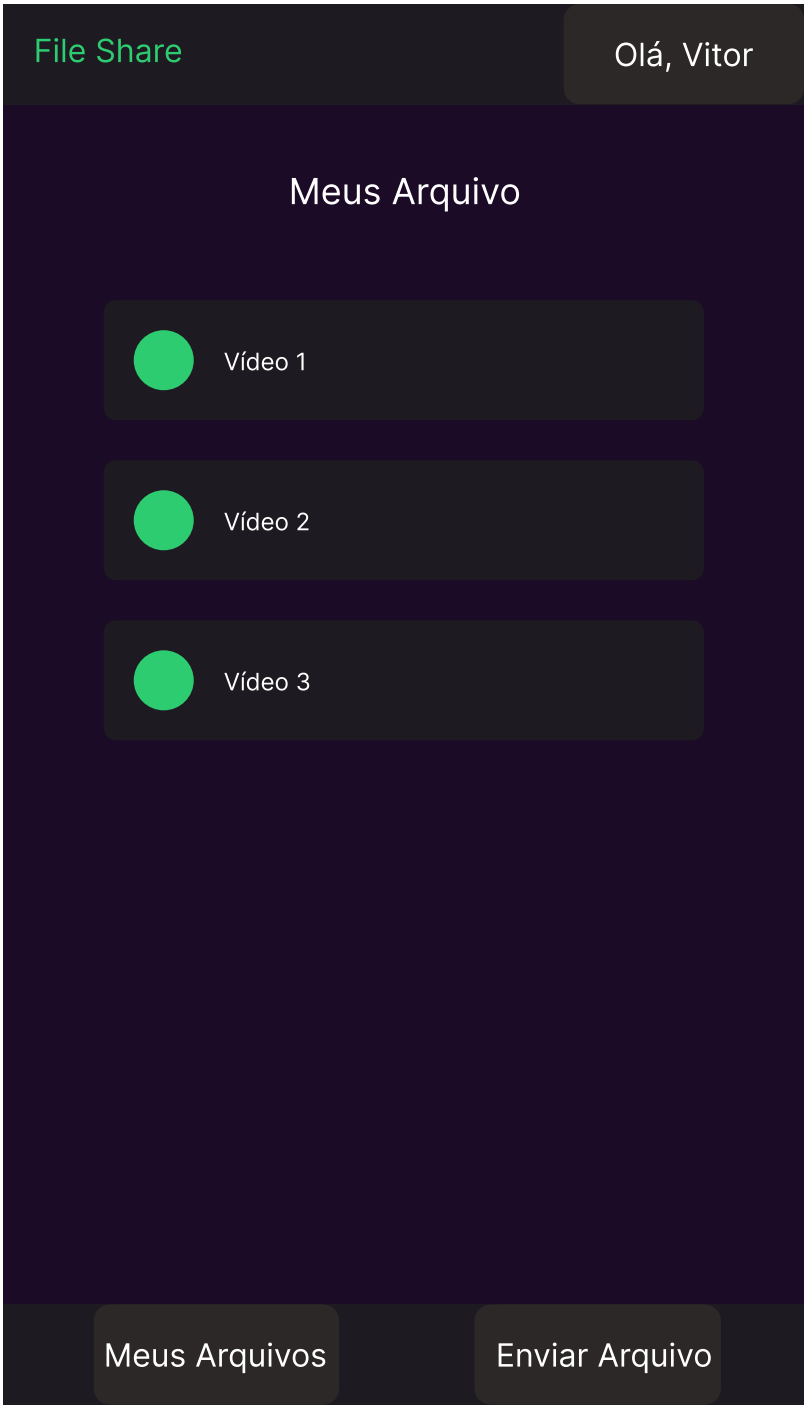
1.2. Tela de Cadastro

Tela de registro de novos usuários.



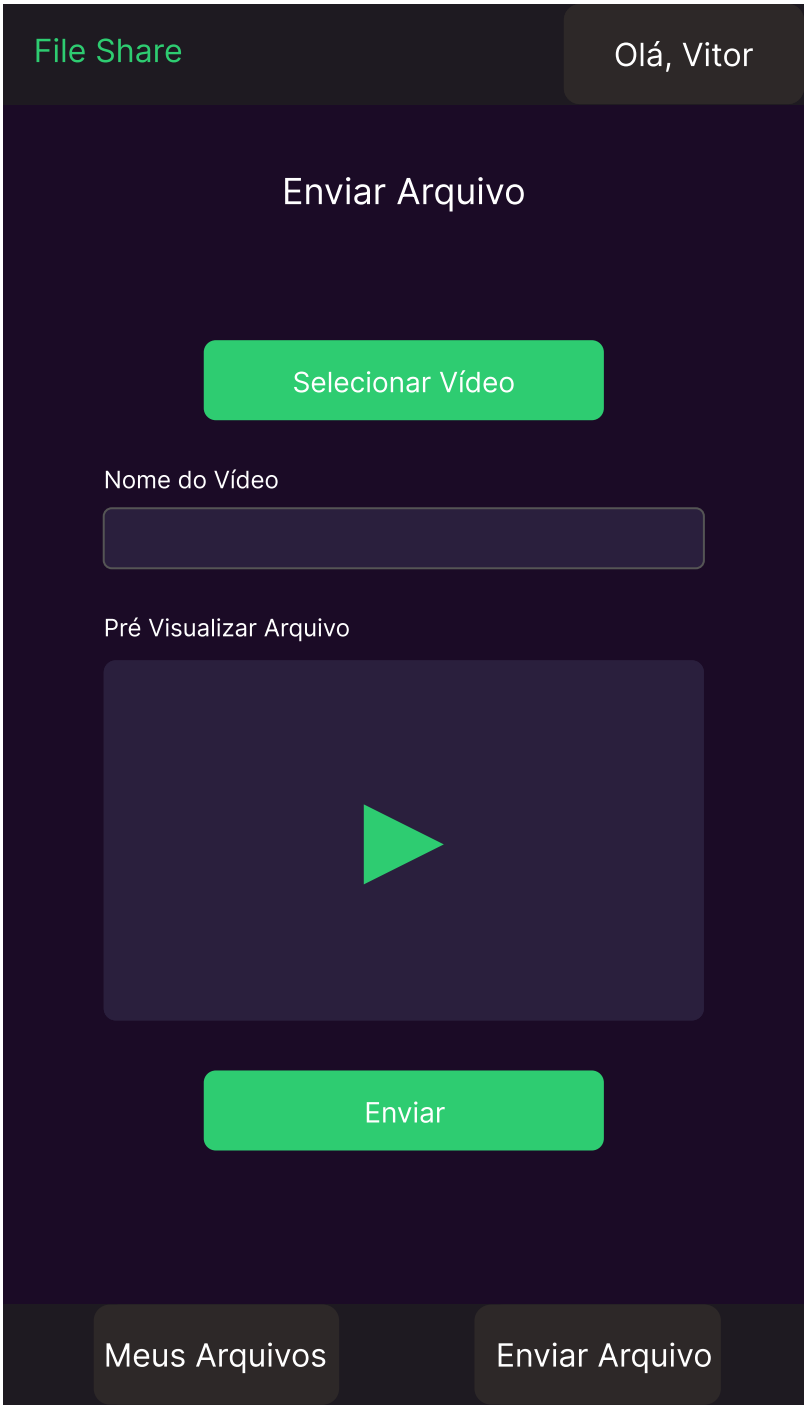
1.3. Tela de Listagem de Arquivos (Meus Arquivos)

Exibe a lista de arquivos enviados pelo usuário.

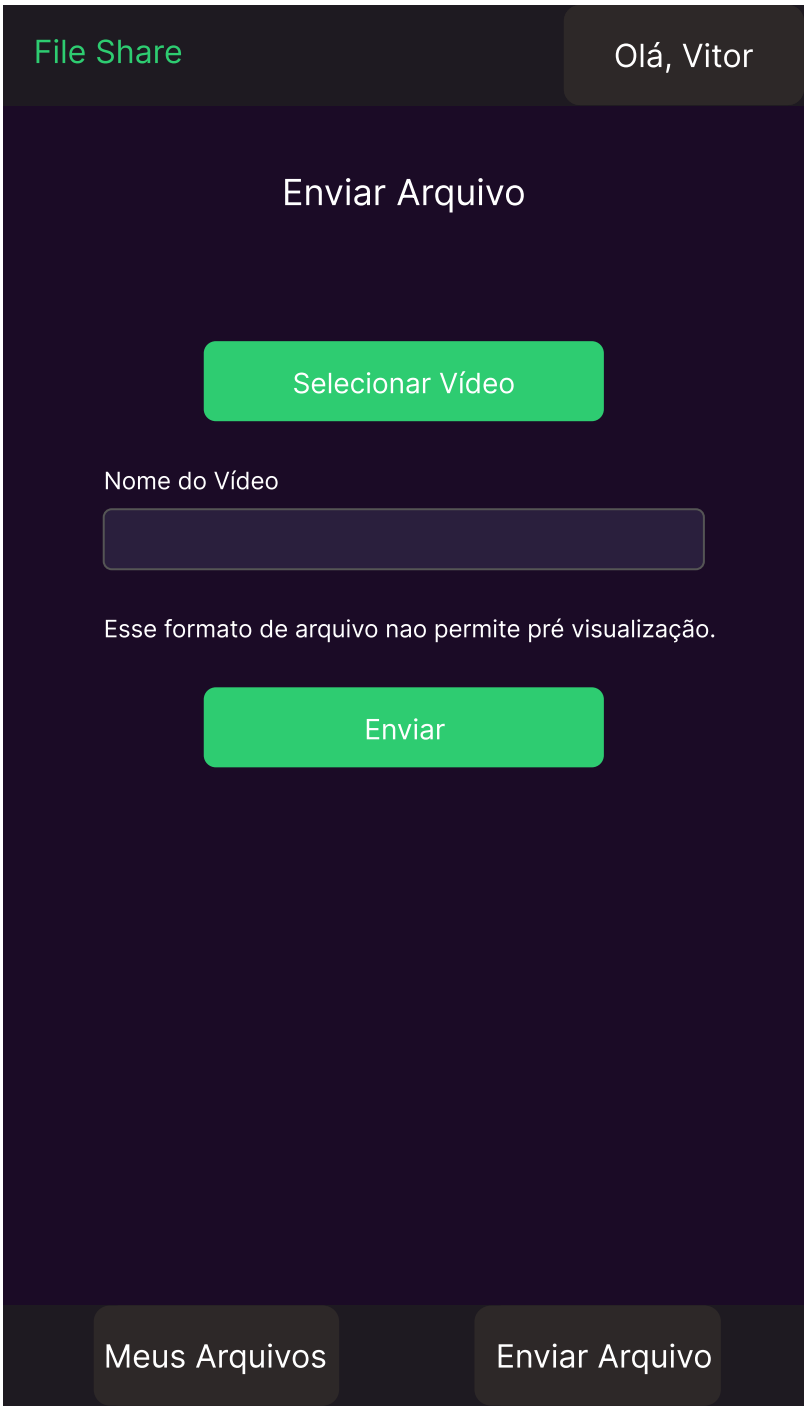


1.4. Tela de Envio de Arquivo

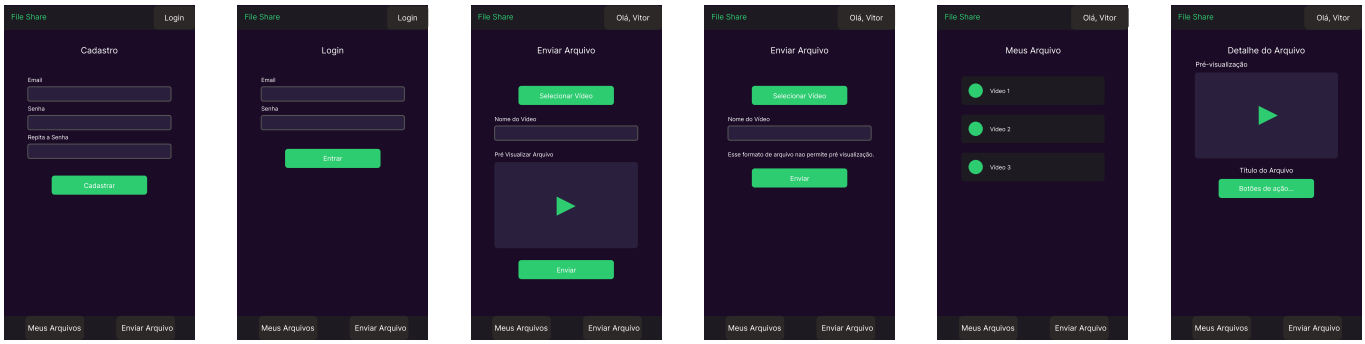
Permite selecionar e enviar um novo arquivo para a plataforma.



1.4.1 Tela de Envio de arquivo sem pré-visualização



1.5. Tela de Visualização de Arquivo

Apresenta os detalhes de um arquivo selecionado e permite sua reprodução.

2. Todas as telas juntas